

Lecture 8: Density Estimation, and Classification

ML/AI for Engineering Applications
ENV 6932

Sara Behdad
Fall 2025

Outline

- MLE
- MAP
- Classification

Maximum Likelihood Estimation (MLE)

- MLE estimates parameters θ based on the principle that if we observe training data D , we should choose the value of θ that makes D most probable.

$$\hat{\theta}^{MLE} = \arg \max_{\theta} P(D|\theta)$$

- **The underlying principle:** we are more likely to observe data D if we are in a world where the appearance of this data is highly probable. Therefore, we should select θ value that maximizes the probability of having observed D .
- Example: flipping a coin
- X : a random variable which can take on either value 1 or 0, and $\theta = P(X = 1)$
- Flip the coin X a number of times to produce training data D , (we observe $X = 1$ a total of α_1 times, and $X = 0$ a total of α_0 times).

Maximum Likelihood Estimation

- **MLE:** choosing θ to maximize $P(D | \theta)$.
- Write an expression for $P(D | \theta)$, or equivalently $P(\alpha_1, \alpha_0 | \theta)$ in terms of θ , then find an algorithm that chooses θ that maximizes this quantity.
- **Case 1:** if data D consists of just one coin flip, then $P(D | \theta) = \theta$ if $X = 1$, and $P(D | \theta) = (1 - \theta)$ if $X = 0$.
- **Case 2:** if a set of i.i.d. coin flips such as $D = \{1, 1, 0, 1, 0\}$, then we can easily calculate $P(D | \theta)$ by multiplying the probabilities of each individual coin flip

Maximum Likelihood Estimation

- **MLE:** choosing θ to maximize $P(D | \theta)$.
- Write an expression for $P(D | \theta)$, or equivalently $P(\alpha_1, \alpha_0 | \theta)$ in terms of θ , then find an algorithm that chooses θ that maximizes this quantity.
- **Case 1:** if data D consists of just one coin flip, then $P(D | \theta) = \theta$ if $X = 1$, and $P(D | \theta) = (1 - \theta)$ if $X = 0$.
- **Case 2:** if a set of i.i.d. coin flips such as $D = \{1, 1, 0, 1, 0\}$, then we can easily calculate $P(D | \theta)$ by multiplying the probabilities of each individual coin flip

$$P(D = \{1, 1, 0, 1, 0\} | \theta) = \theta \cdot \theta \cdot (1 - \theta) \cdot \theta \cdot (1 - \theta) = \theta^3 \cdot (1 - \theta)^2$$

Maximum Likelihood Estimation

- Generally, if the total number of observed times α_1 when $X = 1$ and the number of times α_0 that $X = 0$, we have

$$P(D = \langle \alpha_1, \alpha_0 \rangle | \theta) = \theta^{\alpha_1} (1 - \theta)^{\alpha_0}$$

- $P(D | \theta)$ is often called the data likelihood, or the data likelihood function.

$$\arg \max_{\theta} P(D | \theta) = \arg \max_{\theta} \ln P(D | \theta)$$

Maximum Likelihood Estimation

$$\begin{aligned}\frac{\partial \ell(\theta)}{\partial \theta} &= \frac{\partial \ln P(D|\theta)}{\partial \theta} \\ &= \frac{\partial \ln[\theta^{\alpha_1} (1-\theta)^{\alpha_0}]}{\partial \theta} \\ &= \frac{\partial [\alpha_1 \ln \theta + \alpha_0 \ln(1-\theta)]}{\partial \theta} \\ &= \alpha_1 \frac{\partial \ln \theta}{\partial \theta} + \alpha_0 \frac{\partial \ln(1-\theta)}{\partial \theta} \\ &= \alpha_1 \frac{\partial \ln \theta}{\partial \theta} + \alpha_0 \frac{\partial \ln(1-\theta)}{\partial(1-\theta)} \cdot \frac{\partial(1-\theta)}{\partial \theta} \\ \frac{\partial \ell(\theta)}{\partial \theta} &= \alpha_1 \frac{1}{\theta} + \alpha_0 \frac{1}{(1-\theta)} \cdot (-1)\end{aligned}$$

Maximum Likelihood Estimation

$$\begin{aligned}0 &= \alpha_1 \frac{1}{\theta} - \alpha_0 \frac{1}{1-\theta} \\ \alpha_0 \frac{1}{1-\theta} &= \alpha_1 \frac{1}{\theta} \\ \alpha_0 \theta &= \alpha_1 (1-\theta) \\ (\alpha_1 + \alpha_0) \theta &= \alpha_1 \\ \theta &= \frac{\alpha_1}{\alpha_1 + \alpha_0}\end{aligned}$$

Maximum a Posteriori Probability Estimation (MAP)

- choose the value of θ that is most probable, given the observed data D and our prior assumptions summarized by $P(\theta)$.

$$\hat{\theta}^{MAP} = \arg \max_{\theta} P(\theta|D)$$

- Using Bayes rule

$$\hat{\theta}^{MAP} = \arg \max_{\theta} P(\theta|D) = \arg \max_{\theta} \frac{P(D|\theta)P(\theta)}{P(D)}$$

- given that $P(D)$ does not depend on θ

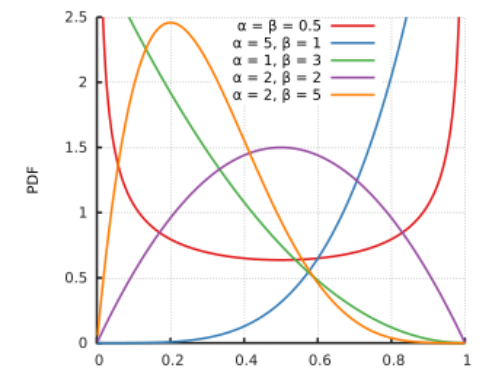
$$\hat{\theta}^{MAP} = \arg \max_{\theta} P(\theta|D) = \arg \max_{\theta} P(D|\theta)P(\theta)$$

Maximum a Posteriori Probability Estimation

- Comparing this to MLE: The only difference is the extra $P(\theta)$.
- Let's assume a Beta distribution for prior:

$$P(\theta) = \text{Beta}(\beta_0, \beta_1) = \frac{\theta^{\beta_1-1} (1-\theta)^{\beta_0-1}}{B(\beta_0, \beta_1)}$$

- β_0 and β_1 are parameters, choosing β_0 and β_1 corresponds to choosing the number of imaginary examples γ_0 and γ_1 in Algorithm 2.



Maximum a Posteriori Probability Estimation

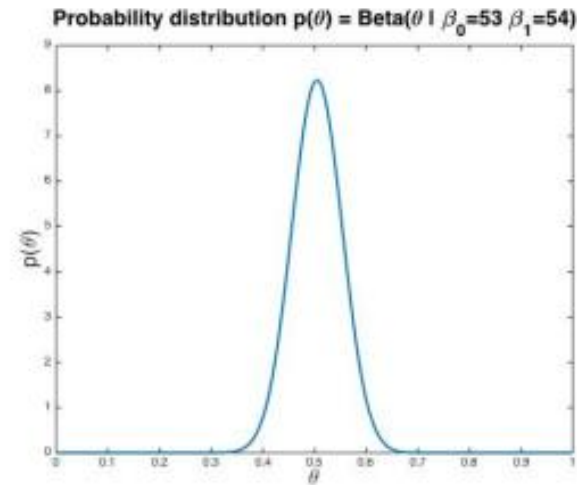
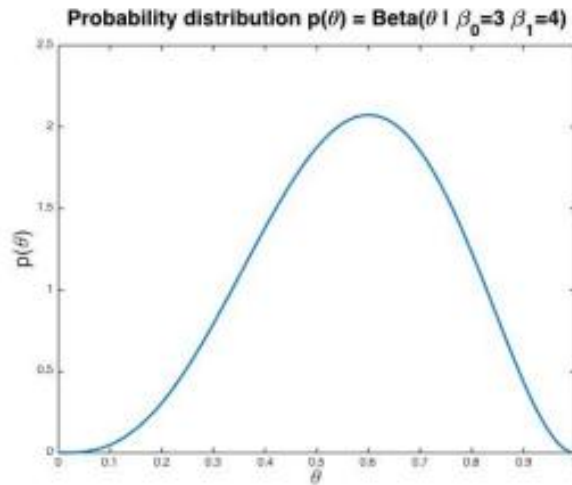
$$\begin{aligned}\hat{\theta}^{MAP} &= \arg \max_{\theta} P(D|\theta)P(\theta) \\ &= \arg \max_{\theta} \theta^{\alpha_1} (1-\theta)^{\alpha_0} \frac{\theta^{\beta_1-1} (1-\theta)^{\beta_0-1}}{B(\beta_0, \beta_1)} \\ &= \arg \max_{\theta} \frac{\theta^{\alpha_1+\beta_1-1} (1-\theta)^{\alpha_0+\beta_0-1}}{B(\beta_0, \beta_1)} \\ &= \arg \max_{\theta} \theta^{\alpha_1+\beta_1-1} (1-\theta)^{\alpha_0+\beta_0-1}\end{aligned}$$

- How can we solve for θ that maximizes the above expression?
- Using MLE, if we substitute $(\alpha_1 + \beta_1 - 1)$ for α_1 , and substitute $(\alpha_0 + \beta_0 - 1)$ for α_0

$$\hat{\theta}^{MAP} = \arg \max_{\theta} P(D|\theta)P(\theta) = \frac{(\alpha_1 + \beta_1 - 1)}{(\alpha_1 + \beta_1 - 1) + (\alpha_0 + \beta_0 - 1)}$$

Example of Beta prior and posterior distribution

- Prior (left) and posterior (right) probability distributions on θ in a MAP estimate for θ .



Discrete Valued Random Variables with Many Values

- A common approach to characterizing n-valued random variables is to use a generalization of the Bernoulli distribution called a Categorical distribution.
- Example: a six-sided die as a random variable X that can take on the values 1 through 6, $\theta = \langle \theta_1 \dots \theta_6 \rangle$, where the

$$\theta_i = P(X = i).$$

$$L(\theta) = P(D|\theta) = \theta_1^{\alpha_1} \theta_2^{\alpha_2} \dots \theta_n^{\alpha_n}$$

α_i : number of times when the value $X = i$ was observed in the data.

- MLE:

$$\hat{\theta}_i^{MLE} = \frac{\alpha_i}{\alpha_1 + \dots + \alpha_n}$$

MAP estimation for Categorical distribution

- we use the conjugate prior for the Categorical distribution, which is called the Dirichlet distribution.
- Dirichlet is a generalization of Beta distribution:

$$P(\theta_1, \dots, \theta_n) = \frac{\theta_1^{(\beta_1-1)} \theta_2^{(\beta_2-1)} \dots \theta_n^{(\beta_n-1)}}{B(\beta_1, \dots, \beta_n)}$$

MAP estimation for each θ_i

$$\hat{\theta}_i^{MAP} = \frac{(\alpha_i + \beta_i - 1)}{(\alpha_1 + \beta_1 - 1) + \dots + (\alpha_n + \beta_n - 1)}$$

References

- Tom M. Mitchell, Machine Learning, 2017, Chapter 2: Estimating Probabilities, http://www.cs.cmu.edu/~tom/mlbook/Joint_MLE_MAP.pdf
- <https://www.javatpoint.com/linear-regression-vs-logistic-regression-in-machine-learning>
- <https://medium.com/mlearning-ai/multiple-linear-regression-fundamentals-and-modeling-in-python-60db7095deff>
- <https://towardsdatascience.com/cross-validation-explained-evaluating-estimator-performance-e51e5430ff85>
- <https://towardsdatascience.com/evaluation-metrics-model-selection-in-linear-regression-73c7573208be>
- IBM Machine Learning with Python: A Practical Introduction

Classification

Classification

- Naïve Bayes
- Perceptron Algorithm
- K-Nearest Neighbors
- Decision Tree
- Random Forest
- Support Vector Machine

Definition of Classification

- The process of recognizing, understanding, and grouping objects into preset categories or “sub-populations.”
- Machine learning uses a variety of algorithms to classify future datasets into categories using pre-categorized training datasets.



The Naive Bayes algorithm

- A classification algorithm based on Bayes rule and a set of conditional independence assumptions.
- The goal: learning $P(Y|X)$ where $X = \langle X_1, \dots, X_n \rangle$,
- Assumption that each X_i is conditionally independent of each of the other X_k s given Y , and also independent of each subset of the other X_k 's given Y .
- The probability that Y will take on its k th possible value, according to Bayes rule:

$$P(Y = y_k | X_1 \dots X_n) = \frac{P(Y = y_k)P(X_1 \dots X_n | Y = y_k)}{\sum_j P(Y = y_j)P(X_1 \dots X_n | Y = y_j)}$$

The Naive Bayes algorithm

- Naïve Bayes Assumption:

$$P(\mathbf{x}|y) = \prod_i P(x_i|y)$$

- Therefore, the fundamental equation for the Naive Bayes classifier is:

$$P(Y = y_k | X_1 \dots X_n) = \frac{P(Y = y_k) \prod_i P(X_i | Y = y_k)}{\sum_j P(Y = y_j) \prod_i P(X_i | Y = y_j)}$$

- which simplifies to the following (since the denominator does not depend on y_k).

$$Y \leftarrow \arg \max_{y_k} P(Y = y_k) \prod_i P(X_i | Y = y_k)$$

Example

- Given the set of attribute, predict the class.

A: attributes

M: mammals

N: non-mammals

Name	Give Birth	Can Fly	Live in Water	Have Legs	Class
human	yes	no	no	yes	mammals
python	no	no	no	no	non-mammals
salmon	no	no	yes	no	non-mammals
whale	yes	no	yes	no	mammals
frog	no	no	sometimes	yes	non-mammals
komodo	no	no	no	yes	non-mammals
bat	yes	yes	no	yes	mammals
pigeon	no	yes	no	yes	non-mammals
cat	yes	no	no	yes	mammals
leopard shark	yes	no	yes	no	non-mammals
turtle	no	no	sometimes	yes	non-mammals
penguin	no	no	sometimes	yes	non-mammals
porcupine	yes	no	no	yes	mammals
eel	no	no	yes	no	non-mammals
salamander	no	no	sometimes	yes	non-mammals
gila monster	no	no	no	yes	non-mammals
platypus	no	no	no	yes	mammals
owl	no	yes	no	yes	non-mammals
dolphin	yes	no	yes	no	mammals
eagle	no	yes	no	yes	non-mammals

Give Birth	Can Fly	Live in Water	Have Legs	Class
yes	no	yes	no	?

Example

A: attributes

M: mammals

N: non-mammals

$$P(A|M) = \frac{6}{7} \times \frac{6}{7} \times \frac{2}{7} \times \frac{2}{7} = 0.06$$

$$P(A|N) = \frac{1}{13} \times \frac{10}{13} \times \frac{3}{13} \times \frac{4}{13} = 0.0042$$

$$P(A|M)P(M) = 0.06 \times \frac{7}{20} = 0.021$$

$$P(A|N)P(N) = 0.004 \times \frac{13}{20} = 0.0027$$

$$P(A|M)P(M) >$$

$$P(A|N)P(N)$$

=> Mammals

Name	Give Birth	Can Fly	Live in Water	Have Legs	Class
human	yes	no	no	yes	mammals
python	no	no	no	no	non-mammals
salmon	no	no	yes	no	non-mammals
whale	yes	no	yes	no	mammals
frog	no	no	sometimes	yes	non-mammals
komodo	no	no	no	yes	non-mammals
bat	yes	yes	no	yes	mammals
pigeon	no	yes	no	yes	non-mammals
cat	yes	no	no	yes	mammals
leopard shark	yes	no	yes	no	non-mammals
turtle	no	no	sometimes	yes	non-mammals
penguin	no	no	sometimes	yes	non-mammals
porcupine	yes	no	no	yes	mammals
eel	no	no	yes	no	non-mammals
salamander	no	no	sometimes	yes	non-mammals
gila monster	no	no	no	yes	non-mammals
platypus	no	no	no	yes	mammals
owl	no	yes	no	yes	non-mammals
dolphin	yes	no	yes	no	mammals
eagle	no	yes	no	yes	non-mammals

Give Birth	Can Fly	Live in Water	Have Legs	Class
yes	no	yes	no	?

Note:

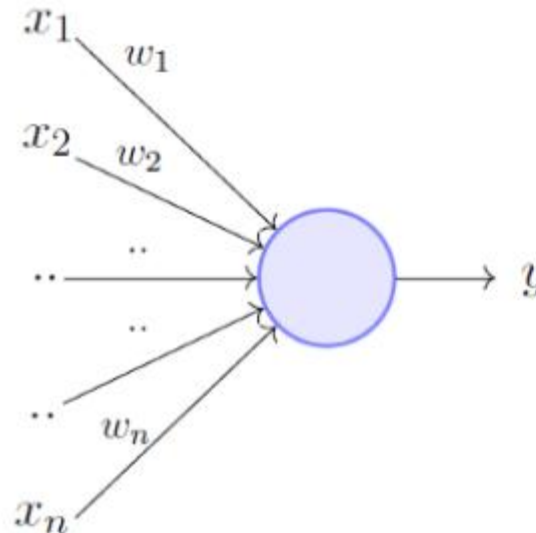
- In practice, some of these counts can be zero
- Fix this by adding “virtual” counts:

$$P(X_i = u|Y = v) = \frac{\text{Count}(X_i = u \wedge Y = v) + 1}{\text{Count}(Y = v) + 2}$$

- This is called Smoothing

Perceptron Learning Algorithm

- Originally proposed by Frank Rosenblatt in 1943, Later developed and carefully analyzed by Minsky and Papert in 1969.
- The perceptron model takes an input, aggregates it (weighted sum) and returns 1 only if the aggregated sum is more than some threshold else returns 0.



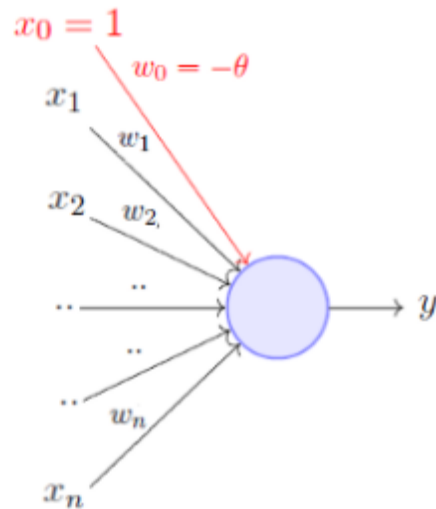
$$y = 1 \quad \text{if} \quad \sum_{i=1}^n w_i * x_i \geq \theta$$
$$= 0 \quad \text{if} \quad \sum_{i=1}^n w_i * x_i < \theta$$

Rewriting the above,

$$y = 1 \quad \text{if} \quad \sum_{i=1}^n w_i * x_i - \theta \geq 0$$
$$= 0 \quad \text{if} \quad \sum_{i=1}^n w_i * x_i - \theta < 0$$

Perceptron Learning Algorithm

- Let's rewrite the threshold and make it a constant input with a variable weight, we will have:



A more accepted convention,

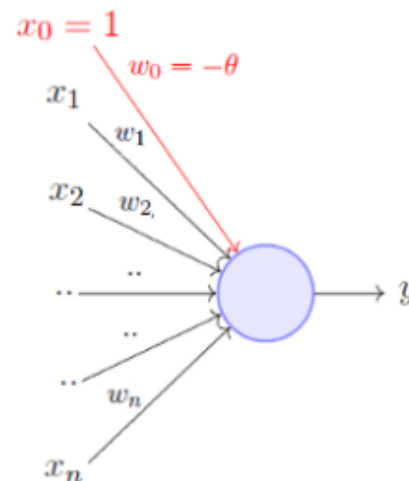
$$y = 1 \quad \text{if} \quad \sum_{i=0}^n w_i * x_i \geq 0$$

$$= 0 \quad \text{if} \quad \sum_{i=0}^n w_i * x_i < 0$$

where, $x_0 = 1$ and $w_0 = -\theta$

Perceptron Learning Algorithm

- A single perceptron can only be used to implement linearly separable functions.
- It takes both real and boolean inputs and associates a set of weights to them, along with a bias (the threshold theta).



A more accepted convention,

$$y = 1 \quad \text{if} \quad \sum_{i=0}^n w_i * x_i \geq 0$$

$$= 0 \quad \text{if} \quad \sum_{i=0}^n w_i * x_i < 0$$

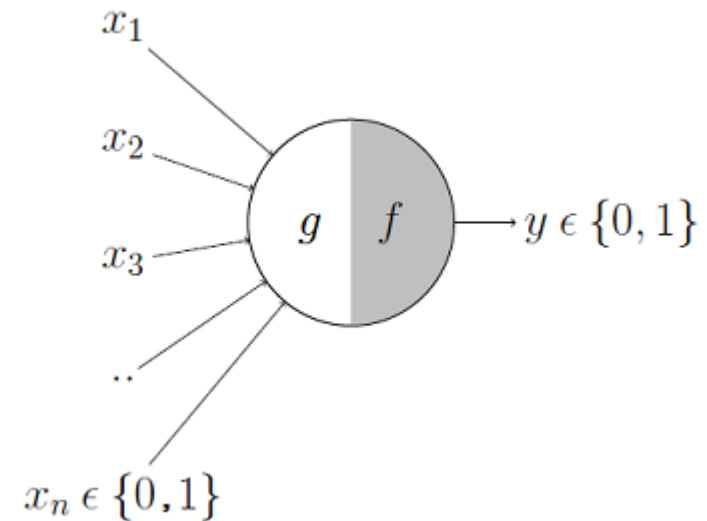
where, $x_0 = 1$ and $w_0 = -\theta$

McCulloch-Pitts Neuron

- The first part, ***g*** takes an input (ahem dendrite ahem), performs an aggregation and based on the aggregated value the second part, ***f*** makes a decision.

$$g(x_1, x_2, x_3, \dots, x_n) = g(\mathbf{x}) = \sum_{i=1}^n x_i$$

$$y = f(g(\mathbf{x})) = \begin{cases} 1 & \text{if } g(\mathbf{x}) \geq \theta \\ 0 & \text{if } g(\mathbf{x}) < \theta \end{cases}$$



Perceptron vs McCulloch-Pitts Neuron

McCulloch Pitts Neuron
(assuming no inhibitory inputs)

$$y = 1 \quad \text{if } \sum_{i=0}^n x_i \geq 0$$
$$= 0 \quad \text{if } \sum_{i=0}^n x_i < 0$$

Perceptron

$$y = 1 \quad \text{if } \sum_{i=0}^n w_i * x_i \geq 0$$
$$= 0 \quad \text{if } \sum_{i=0}^n w_i * x_i < 0$$

- What kind of functions can be implemented using a *perceptron*?

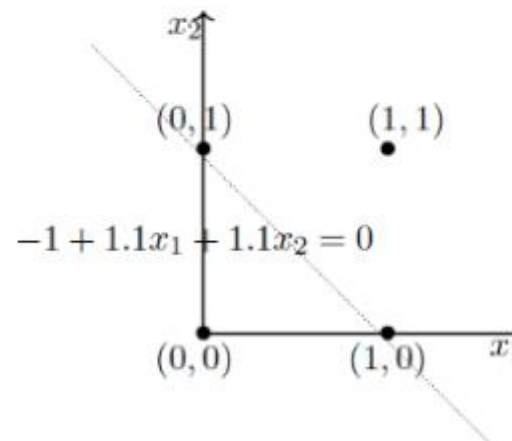
OR Function Using A Perceptron

- Example: Let's use a perceptron to learn an OR function.
- How to learn weights?

x_1	x_2	OR	
0	0	0	$w_0 + \sum_{i=1}^2 w_i x_i < 0$
1	0	1	$w_0 + \sum_{i=1}^2 w_i x_i \geq 0$
0	1	1	$w_0 + \sum_{i=1}^2 w_i x_i \geq 0$
1	1	1	$w_0 + \sum_{i=1}^2 w_i x_i \geq 0$

$$\begin{aligned}w_0 + w_1 \cdot 0 + w_2 \cdot 0 < 0 &\implies w_0 < 0 \\w_0 + w_1 \cdot 0 + w_2 \cdot 1 \geq 0 &\implies w_2 > -w_0 \\w_0 + w_1 \cdot 1 + w_2 \cdot 0 \geq 0 &\implies w_1 > -w_0 \\w_0 + w_1 \cdot 1 + w_2 \cdot 1 \geq 0 &\implies w_1 + w_2 > -w_0\end{aligned}$$

One possible solution is
 $w_0 = -1, w_1 = 1.1, w_2 = 1.1$



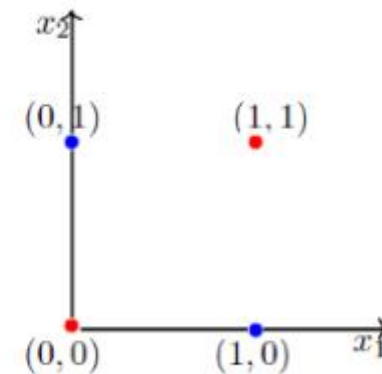
a line that perfectly separates
positive inputs from negative

XOR Function Using A Perceptron

- XOR represents the inequality function, i.e., the output is true if the inputs are not alike otherwise the output is false (must have one or the other but not both)

x_1	x_2	XOR	
0	0	0	$w_0 + \sum_{i=1}^2 w_i x_i < 0$
1	0	1	$w_0 + \sum_{i=1}^2 w_i x_i \geq 0$
0	1	1	$w_0 + \sum_{i=1}^2 w_i x_i \geq 0$
1	1	0	$w_0 + \sum_{i=1}^2 w_i x_i < 0$

$$\begin{aligned}w_0 + w_1 \cdot 0 + w_2 \cdot 0 < 0 &\implies w_0 < 0 \\w_0 + w_1 \cdot 0 + w_2 \cdot 1 \geq 0 &\implies w_2 > -w_0 \\w_0 + w_1 \cdot 1 + w_2 \cdot 0 \geq 0 &\implies w_1 > -w_0 \\w_0 + w_1 \cdot 1 + w_2 \cdot 1 \geq 0 &\implies w_1 + w_2 < -w_0\end{aligned}$$



XOR Function Using A Perceptron

- Note: the fourth equation contradicts the second and the third equation.
- Lesson: there are no *perceptron* solutions for non-linearly separated data. So a **single perceptron** cannot learn to separate the data that are non-linear in nature.

$$w_0 + w_1 \cdot 0 + w_2 \cdot 0 < 0 \implies w_0 < 0$$

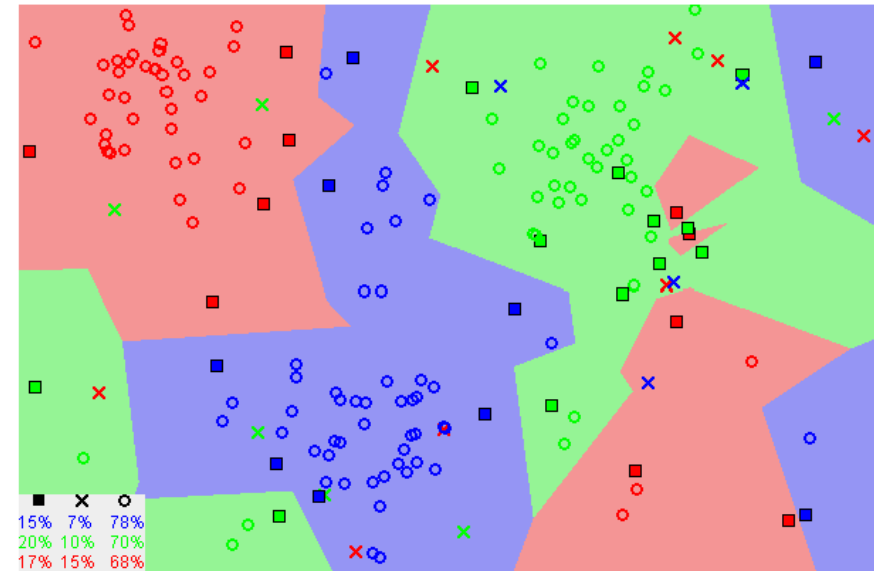
$$w_0 + w_1 \cdot 0 + w_2 \cdot 1 \geq 0 \implies w_2 > -w_0$$

$$w_0 + w_1 \cdot 1 + w_2 \cdot 0 \geq 0 \implies w_1 > -w_0$$

$$w_0 + w_1 \cdot 1 + w_2 \cdot 1 \geq 0 \implies w_1 + w_2 < -w_0$$

K-Nearest Neighbors

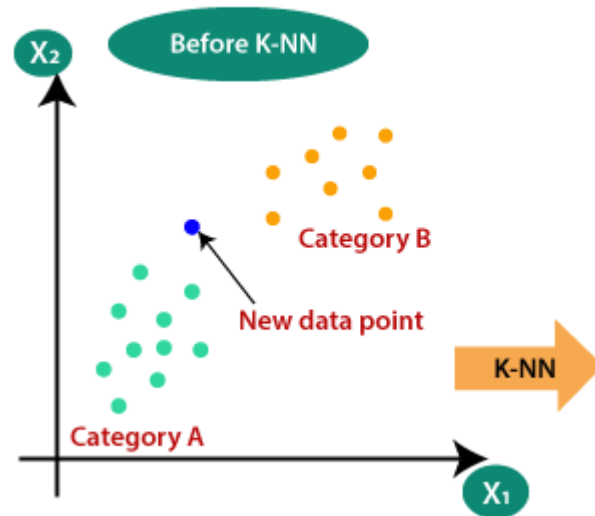
- “Birds of a feather flock together.”
- A non-parametric lazy learning algorithm.



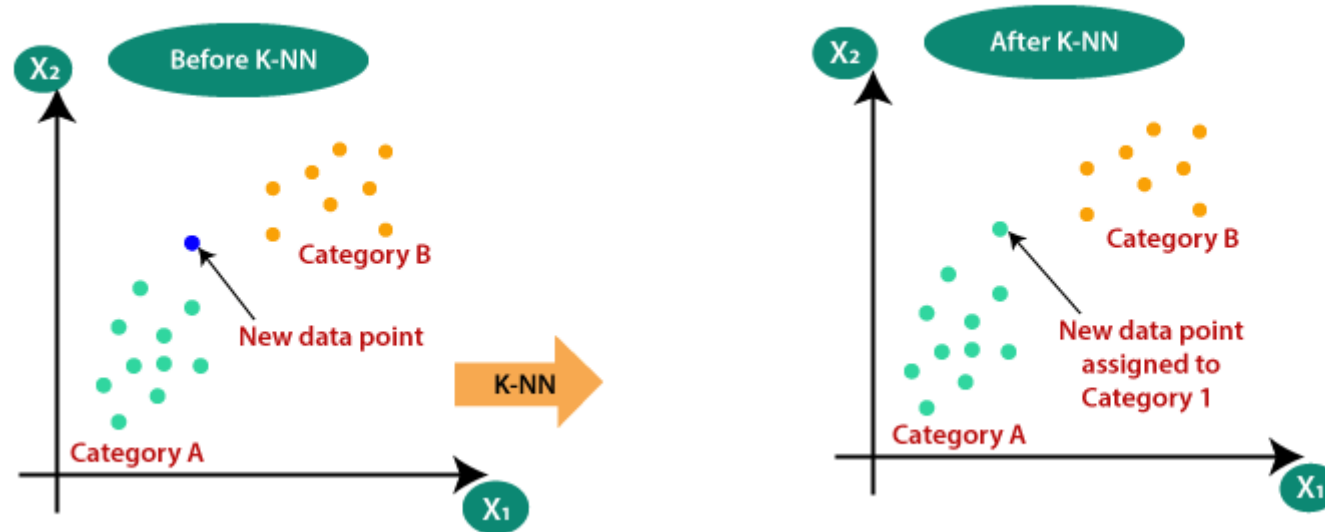
[similar data points typically exist close to each other](#)

K-Nearest Neighbors

- Example: we have two categories, i.e., Category A and Category B. What is the category of a new data point?

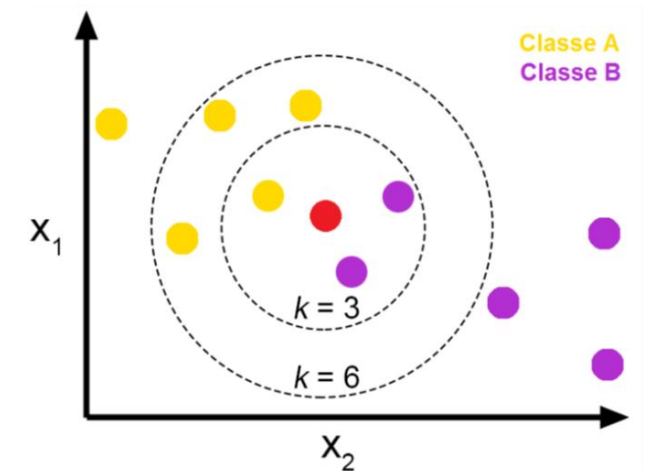


K-Nearest Neighbors



Algorithm

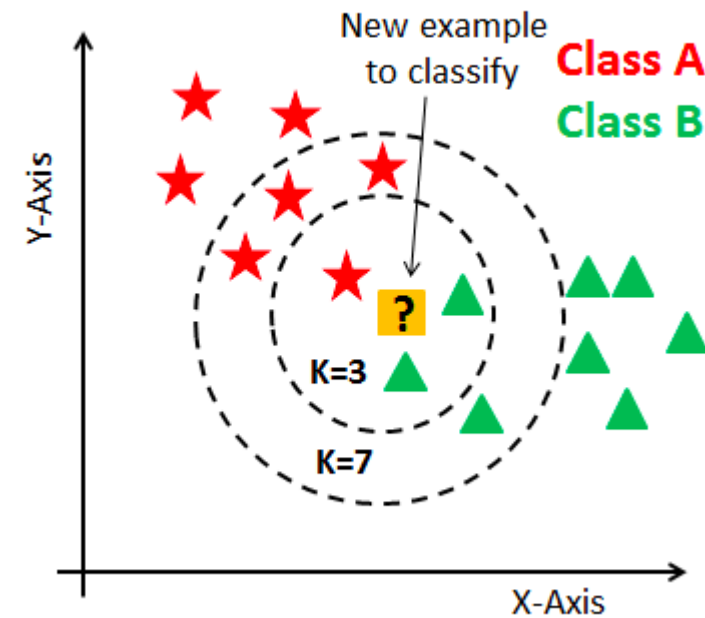
- **Step-1:** Select a value for K
- **Step-2:** Calculate the Euclidean distance of your point from all data points in your training dataset.
- **Step-3:** Take the K nearest neighbors as per the calculated Euclidean distance.
- **Step-4:** Among these k neighbors, count the number of the data points in each category.
- **Step-5:** Assign the new data points to that category for which the number of the neighbor is maximum.



<https://towardsdatascience.com/knn-k-nearest-neighbors-1-a4707b24bd1d>

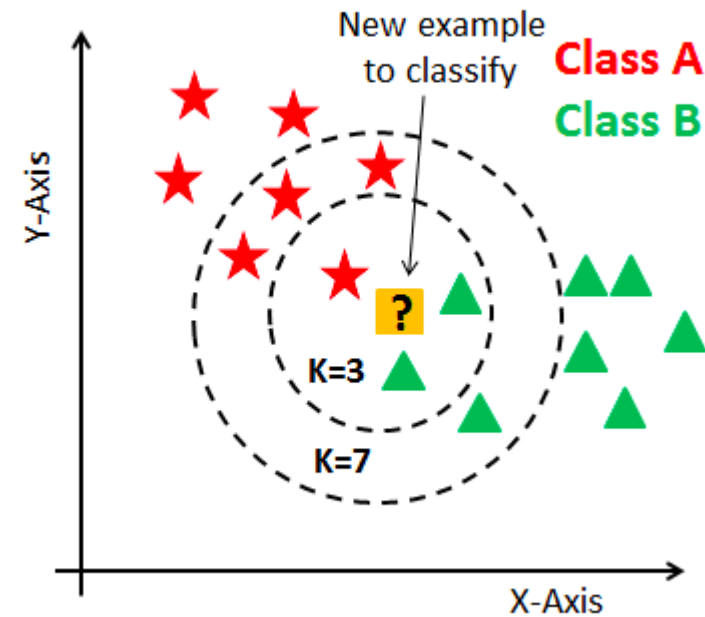
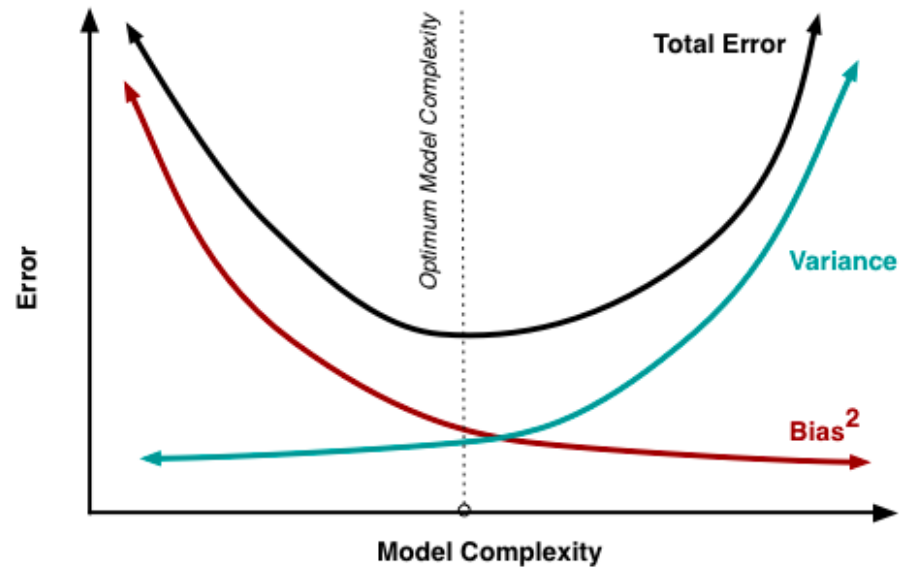
$$\begin{aligned} d(\mathbf{p}, \mathbf{q}) &= d(\mathbf{q}, \mathbf{p}) = \sqrt{(q_1 - p_1)^2 + (q_2 - p_2)^2 + \dots + (q_n - p_n)^2} \\ &= \sqrt{\sum_{i=1}^n (q_i - p_i)^2}. \end{aligned}$$

What is the value of K?

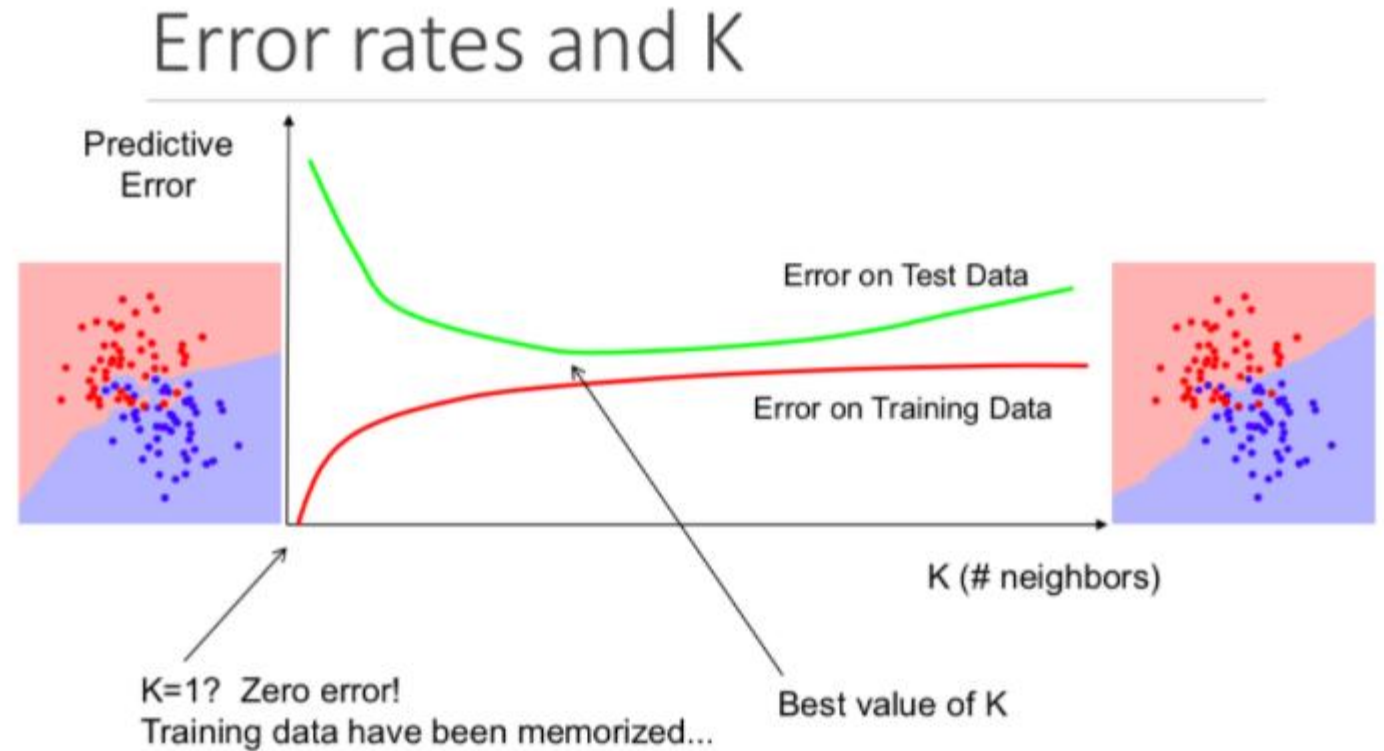


What is the value of K?

- Recall Bias-Variance Tradeoff



- A very low K value will cause the model to perfectly predict training data and poorly predict test data.
- Similarly, a large K value will make the model unnecessarily complex.

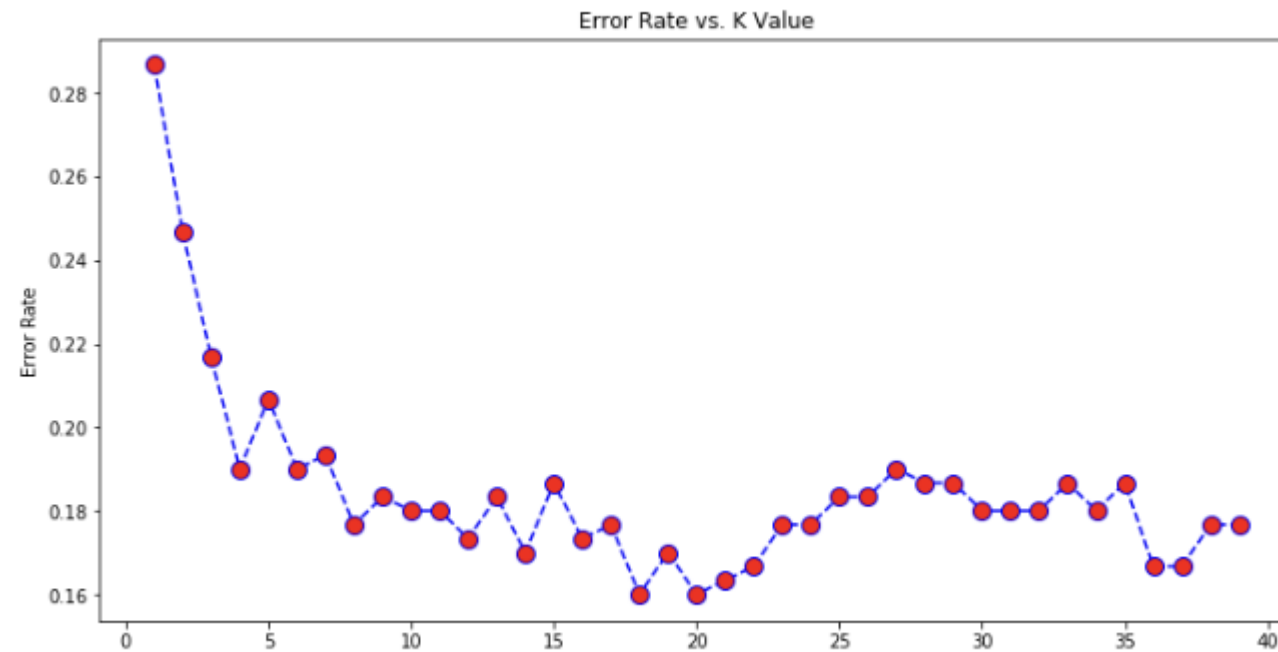


How to choose the best value of K?

- The process of selecting a right value of K is called Hyperparameter Tuning.
- The optimum K depends on the dataset.
- A small value of K means that noise will have a higher influence on the result.
If K is too large, we are under-fitting.
- **Cross Validation Method:** Start with $K=1$, run cross validation (5 to 10 fold), measure the accuracy and keep repeating till the results become consistent.

$K=1, 2, 3...$ As K increases, the error usually decreases, then stabilizes, and then increases again. Pick the optimum K at the beginning of the stable zone. This is also called **Elbow Method**.

Elbow Method



Example:

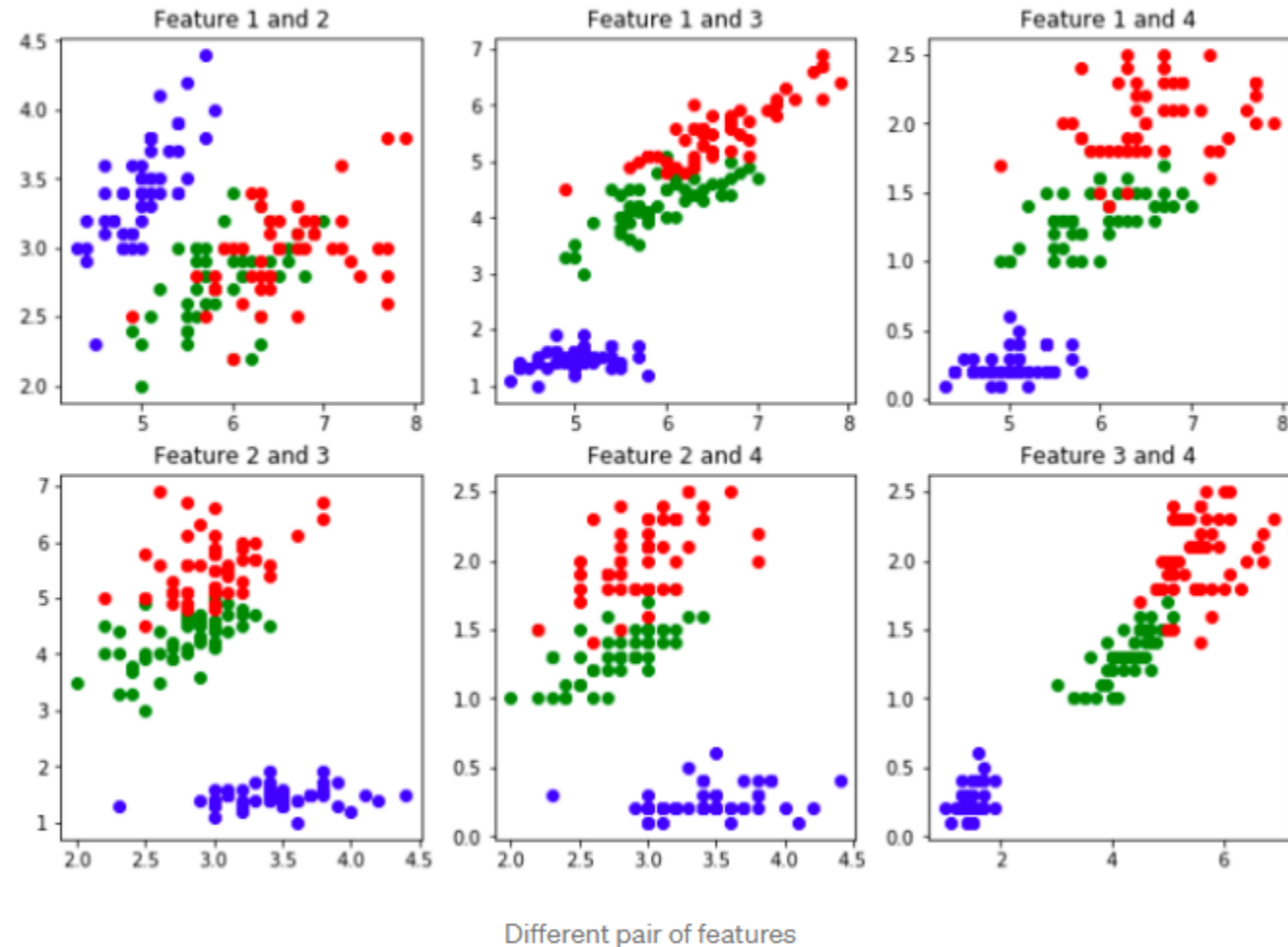
- Iris dataset: 3 classes which represent 3 different kinds of iris. The goal is to classify which iris it is.



Classes	3
Samples per class	50
Samples total	150
Dimensionality	4
Features	real, positive

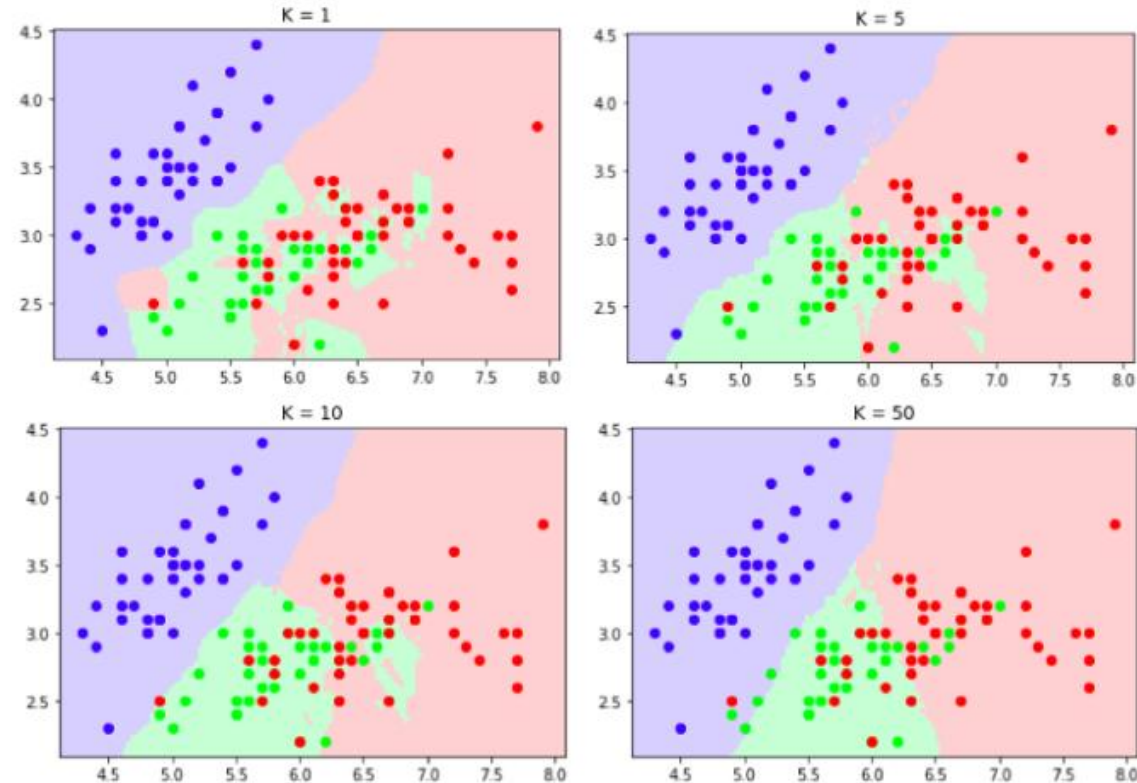
Exploration of 4 Features

- 4 features in this dataset.
- After data exploration, let's choose feature 1 and 2 because it has more overlap.



Visualization of Decision Boundary

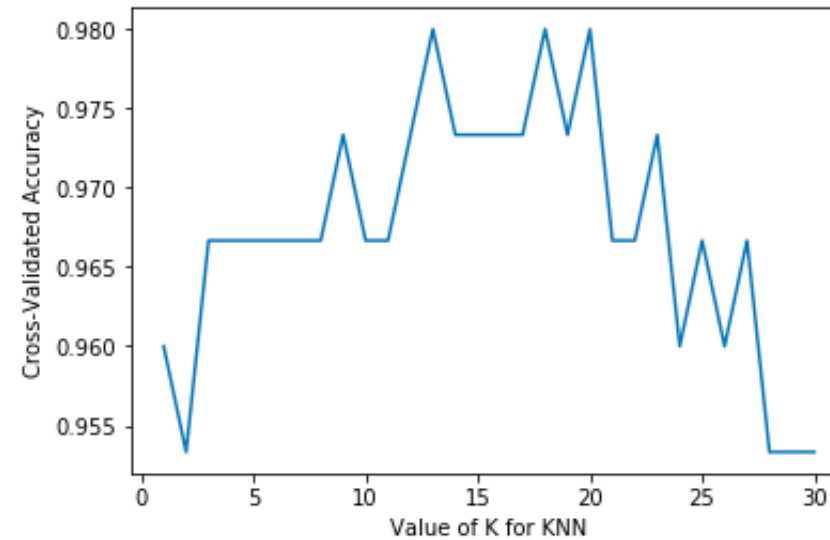
- Test different K and visualize the decision boundary.
- When K is small, some outliers of green label are still green, and outliers of red label are still red.
- When K is larger, the boundary is more consistent and reasonable.



Visualize the result based on different K

Cross-Validation Test

- Choose K around 13 or 20



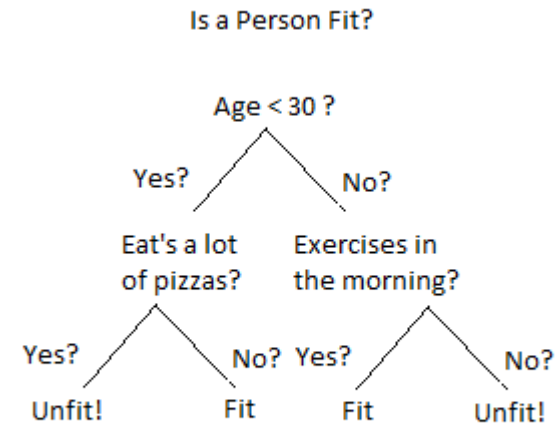
The cross-validation accuracy based on different K

Code is available at the following link:

https://github.com/tzuchi9527/30daysofML/blob/master/Day03_KNN.ipynb

Decision Tree Algorithm

- Decision Trees (DTs) are a non-parametric supervised learning method.
- Used for classification and regression.



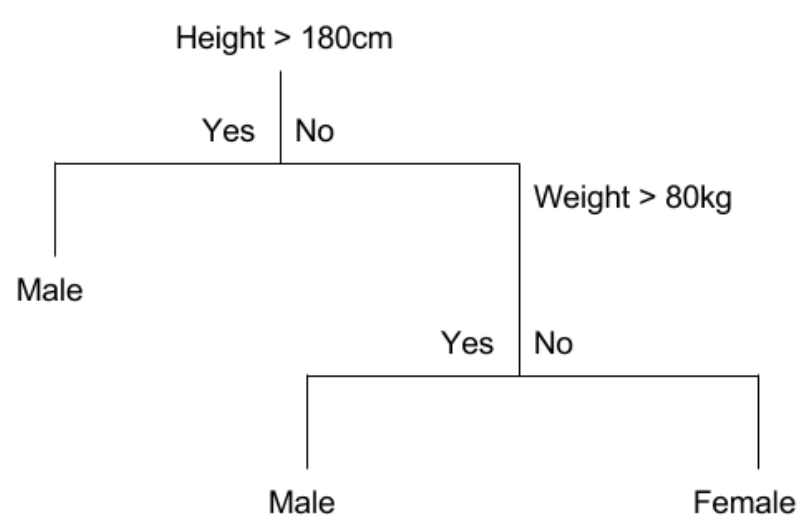
Source: xoriant.com

Elements of Decision Tree

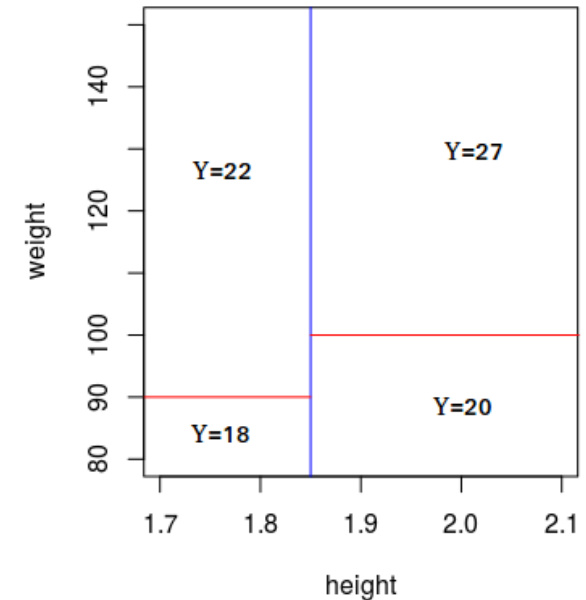
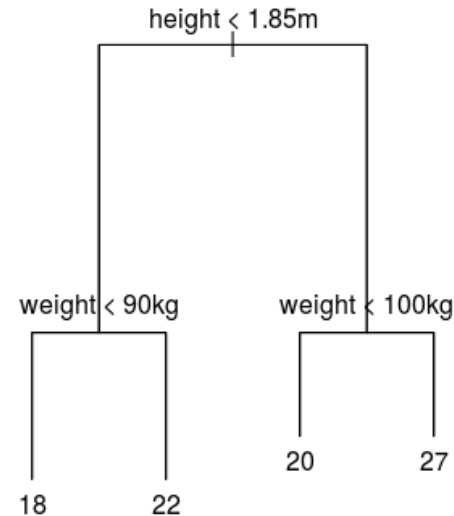
- **Decision Tree consists of:**
- **Nodes:** Test for the value of a certain attribute.
- **Edges/ Branch:** Correspond to the outcome of a test and connect to the next node or leaf.
- **Leaf nodes:** Terminal nodes that predict the outcome (represent class labels or class distribution).

Two Main Types of Decision Trees

- **Classification trees** (Yes/No types)
- **Regression trees** (Continuous data types)



Example of a Classification Tree



Example of Regression Tree

- Classification And Regression Trees or CART

Decision Tree Learning Algorithm

- Select an attribute from the dataset
- Calculate the significance of attribute in splitting of data
- Split data based on the value of the best attribute
- Go to Step 1

Ways to Split a Decision Tree

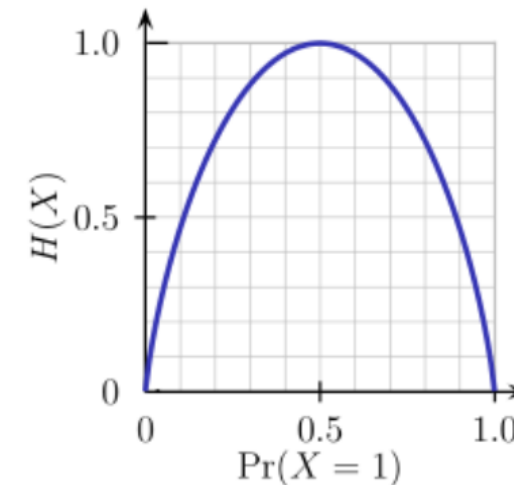
- Continuous Target Variable
 - Reduction in Variance
- Categorical Target Variable
 - Entropy (a measure of randomness)
 - Gini Impurity
 - Information Gain
 - Chi-Square

The Concept of Entropy

- Entropy: the measure of disorder

$$E(S) = \sum_{i=1}^c -p_i \log_2 p_i$$

- p_i : is simply the frequentist probability of an element/class 'i' in our data.
- **Simple example:** we have two classes , a positive class and a negative class. 'i' could be either (+) or (-).
- Suppose we have a total of 100 data points in our dataset with 30 belonging to the positive class and 70 belonging to the negative class:
 - $P(+)= 3/10$
 - $P(-) =7/10$



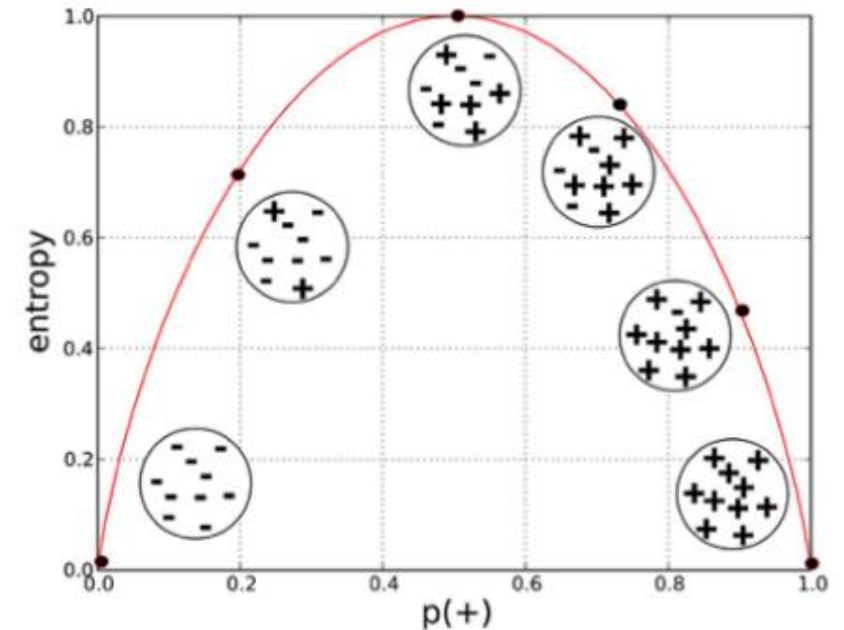
<https://www.kdnuggets.com/2020/01/decision-tree-algorithm-explained.html>

Example of Entropy

- Entropy:

$$-\frac{3}{10} \times \log_2\left(\frac{3}{10}\right) - \frac{7}{10} \times \log_2\left(\frac{7}{10}\right) \approx 0.88$$

- 0.88 is a high entropy , a high level of disorder.
- Entropy is lowest at the extremes (no + or only positive instances)
- Entropy is highest in the middle (bubble is evenly split between positive and negative instances, extreme disorder , no majority).



The x-axis represents the proportion of data points belonging to the positive class, y-axis axis measures their respective entropies.

Information Gain

- How to measure the reduction in disorder?
- A metric to measure the reduction of disorder in our target variable/class given additional information (features/independent variables) about it.
- This is called Information Gain:

$$IG(Y, X) = E(Y) - E(Y|X)$$

Information Gain from X on Y

- Decision trees use entropy and information gain to decide on what features to split their nodes on.

References

- <https://www.analyticsvidhya.com/blog/2020/06/4-ways-split-decision-tree/>
- <https://medium.com/swlh/decision-tree-classification-de64fc4d5aac>
- <https://towardsdatascience.com/entropy-how-decision-trees-make-decisions-2946b9c18c8>